TUGAS PERTEMUAN 5



Dosen Pengampu : Juni Ismail S.kom M.kom

Matkul : Pemrog. Apli. Klien server (Netbeans)

Disusun Oleh

Nama : Dhartati Zai

Nim : 25110016

JURUSAN TEKNIK KOMPUTER

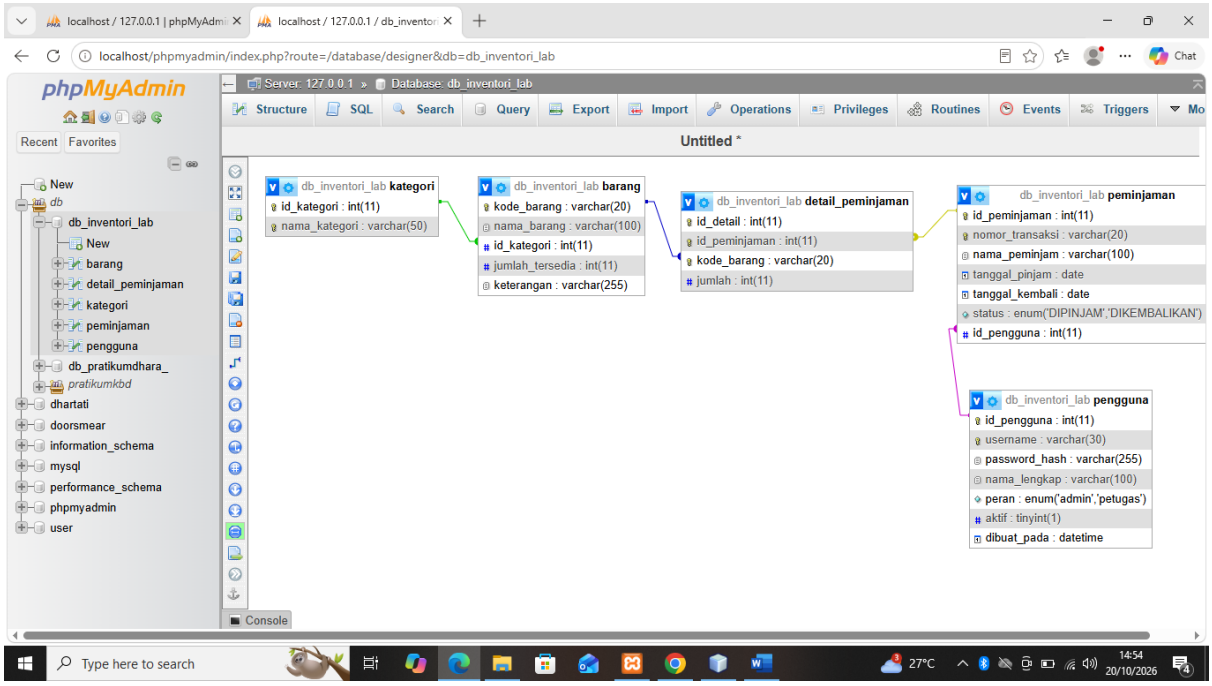
POLITEKNIK BISNIS INDONESIA

TAHUN 2026/2027

A. Pendahuluan dan Tujuan Praktikum

Praktikum ini berfokus pada implementasi, manipulasi, serta pengujian kendali integritas basis data *inventory* laboratorium (db_inventori_lab) menggunakan sistem manajemen basis data MySQL melalui antarmuka phpMyAdmin. Laporan ini merangkum serangkaian eksperimen basis data yang mencakup pemetaan relasi entitas, penerapan kueri ekstraksi data tingkat lanjut (*filtering*, *sorting*, *pattern matching*, *multi-table joins*, dan *aggregations*), operasi modifikasi data (*DML*), hingga simulasi kegagalan sistem akibat pelanggaran aturan integritas (*constraints*).

1. **Designer**
   fitur *Designer* pada phpMyAdmin yang memetakan rancangan logikal database db_inventori_lab secara utuh.

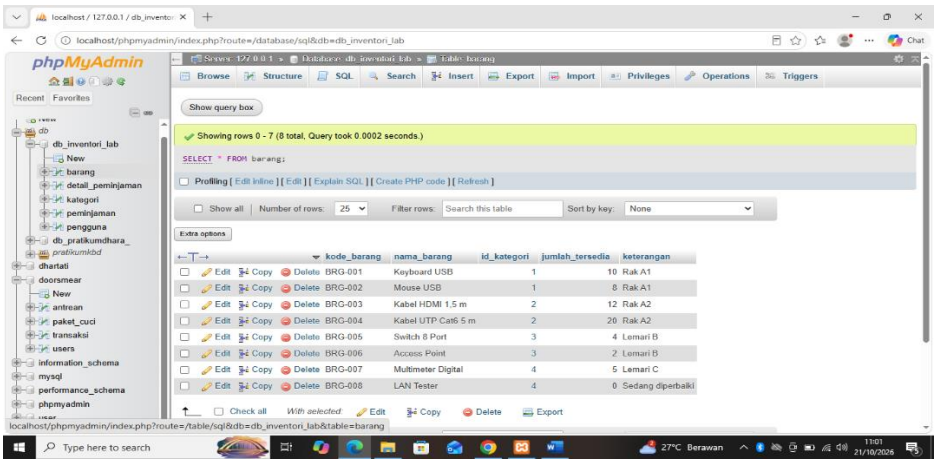


Tampilan ini menyajikan cetak biru arsitektur basis data yang telah menerapkan normalisasi secara optimal. Entitas-entitas utama seperti kategori, barang, pengguna, peminjaman, dan detail_peminjaman dihubungkan menggunakan kunci primer (*primary key*) serta kunci asing (*foreign key*). Pendekatan ini menjamin bahwa seluruh relasi data tetap sinkron, terstruktur dengan baik, serta meminimalisir redundansi data.

2. **Menampilkan Seluruh Record Tabel Barang**
   Perintah SQL

```
SELECT * FROM `barang`
```

kueri di atas berfungsi untuk merestorasi seluruh data mentah dari tabel barang secara menyeluruh. Penggunaan operator bintang (*) memastikan bahwa setiap kolom dan baris yang tersimpan dapat dievaluasi untuk memverifikasi apakah proses *input* data awal telah masuk dengan sukses ke dalam sistem.

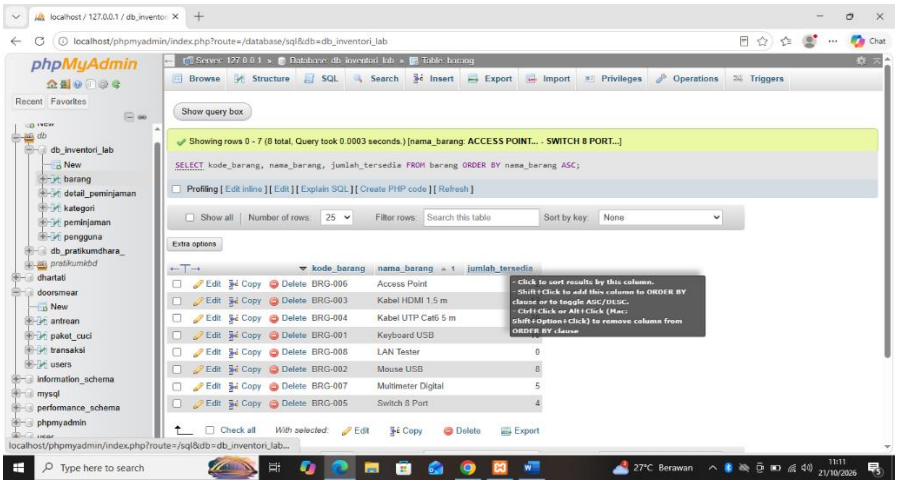


Q1 tersebut berfungsi untuk menampilkan isi kolom, baris atau isi dari tabel apa sudah masuk dengan benar

3. **Memilik kolom tertentu dan mengurutkan sesuai nama**
   Query tersebut hanya akan menampilkan kode barang, nama barang, dan jumlah yang tersedia sesuai dengan urutan nama
   Perintah sql

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
ORDER BY nama_barang ASC;
```

Hasil

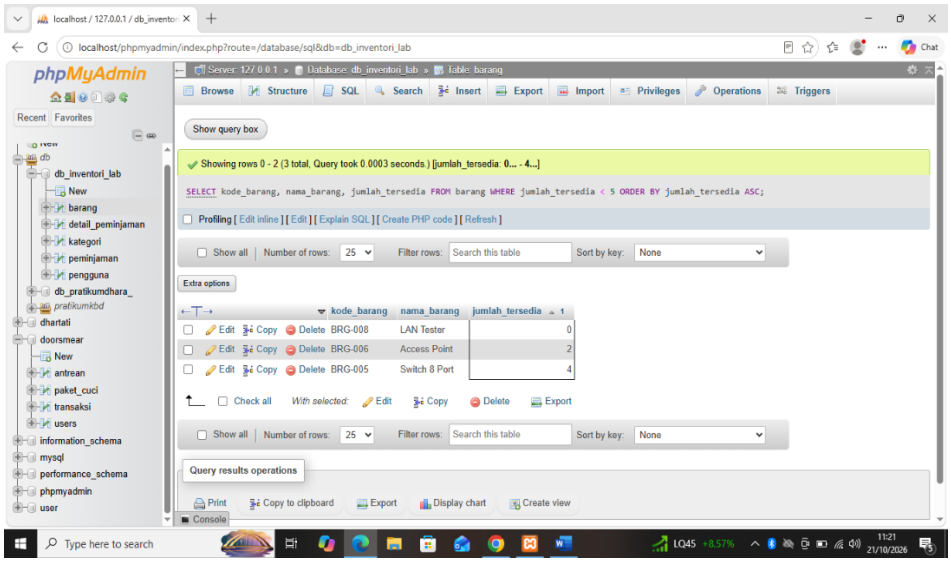


4. **Menyaring dengan where**
   Perintah *WHERE* dipakai untuk menyaring baris, sehingga hanya data yang memenuhi syarat tertentu yang ditampilkan. Pada kueri ini syaratnya adalah *jumlah_tersedia* < *5*, yaitu mencari barang yang stoknya menipis (kurang dari 5 unit).

Perintah sql

```
SELECT kode_barang, nama_barang, jumlah_tersedia
FROM barang
ORDER BY nama_barang ASC;
```

Hasil

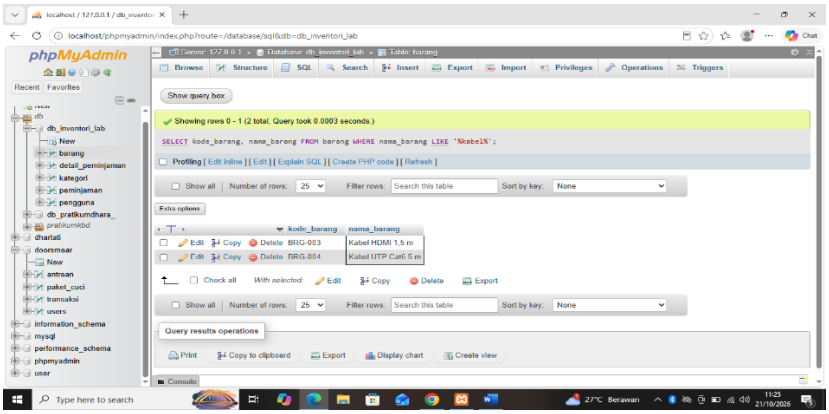


5. **Mencari Sebagian teks dengan like**
   Operator *LIKE* dipakai untuk mencari data berdasarkan sebagian teks (pola), bukan kecocokan penuh.

Perintah sql

```
SELECT kode_barang, nama_barang
FROM barang
WHERE nama_barang LIKE '%kabel%';
```

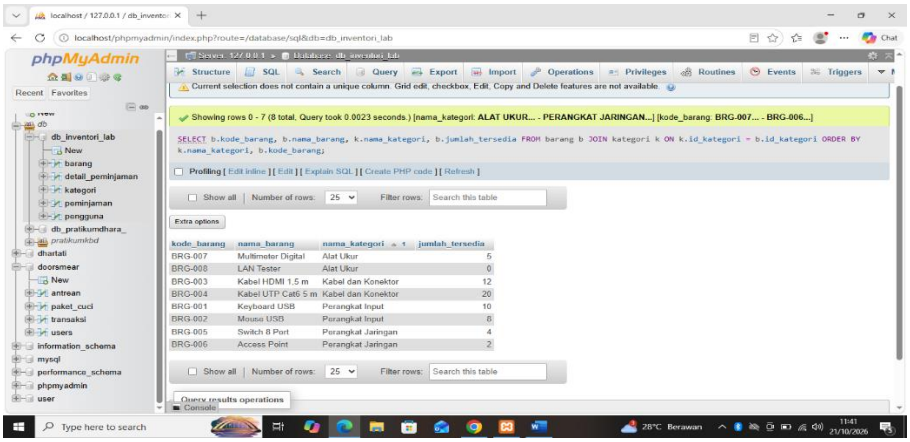
Hasil



6. **Menggabungkan tabel dengan JOIN dan meringkas dengan GROUP BY**
   Pada basis data yang sudah dinormalisasi, data tersebar di beberapa tabel sehingga perlu digabungkan dengan *JOIN*. Tabel barang hanya menyimpan id_kategori berupa angka, sedangkan nama kategorinya ada di tabel kategori.
   Perintah sql

```
SELECT b.kode_barang, b.nama_barang, k.nama_kategori, b.jumlah_tersedia
FROM barang b
JOIN kategori k ON k.id_kategori = b.id_kategori
ORDER BY k.nama_kategori, b.kode_barang;
```

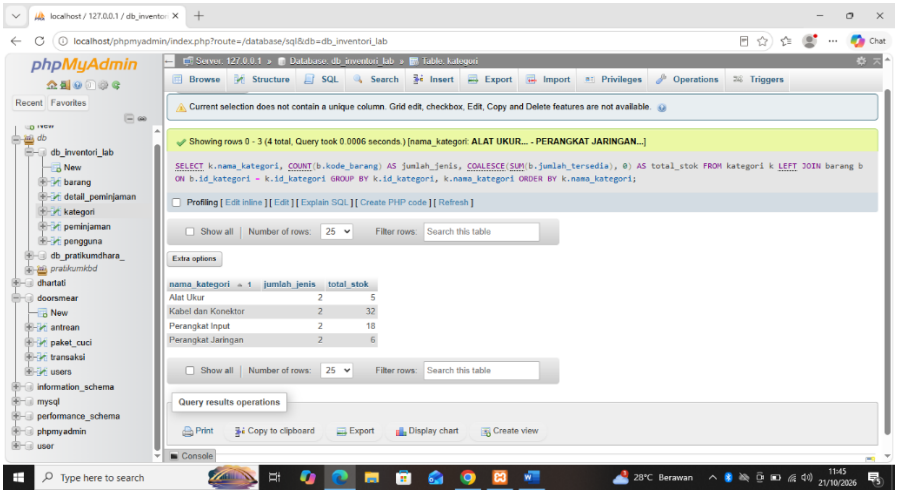
   Hasil



7. **jenis barang dan total stok per kategori**
   Kueri ini meringkas data per kategori memakai fungsi agregasi. *COUNT(b.kode_barang)* menghitung jumlah jenis barang, *SUM(b.jumlah_tersedia)* menjumlahkan stok, dan *GROUP BY* mengelompokkan baris berdasarkan kategori sehingga kedua fungsi dihitung per kelompok.
   perintah sql

```
SELECT k.nama_kategori,
       COUNT(b.kode_barang)              AS jumlah_jenis,
       COALESCE(SUM(b.jumlah_tersedia), 0) AS total_stok
FROM kategori k
LEFT JOIN barang b ON b.id_kategori = k.id_kategori
GROUP BY k.id_kategori, k.nama_kategori
ORDER BY k.nama_kategori;
```

   Hasil



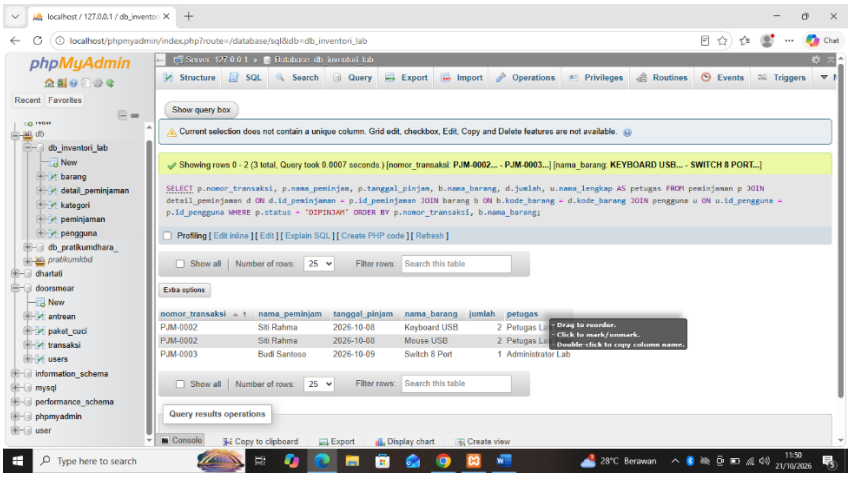
8. **rincian peminjaman yang masih berjalan (4 tabel sekaligus)**

Kueri ini menggabungkan empat tabel sekaligus untuk menampilkan rincian peminjaman yang masih berjalan.

perintah sql

```
SELECT p.nomor_transaksi, p.nama_peminjam, p.tanggal_pinjam,
       b.nama_barang, d.jumlah, u.nama_lengkap AS petugas
FROM peminjaman p
JOIN detail_peminjaman d ON d.id_peminjaman = p.id_peminjaman
JOIN barang b            ON b.kode_barang   = d.kode_barang
JOIN pengguna u          ON u.id_pengguna   = p.id_pengguna
WHERE p.status = 'DIPINJAM'
ORDER BY p.nomor_transaksi, b.nama_barang;
```

Hasil



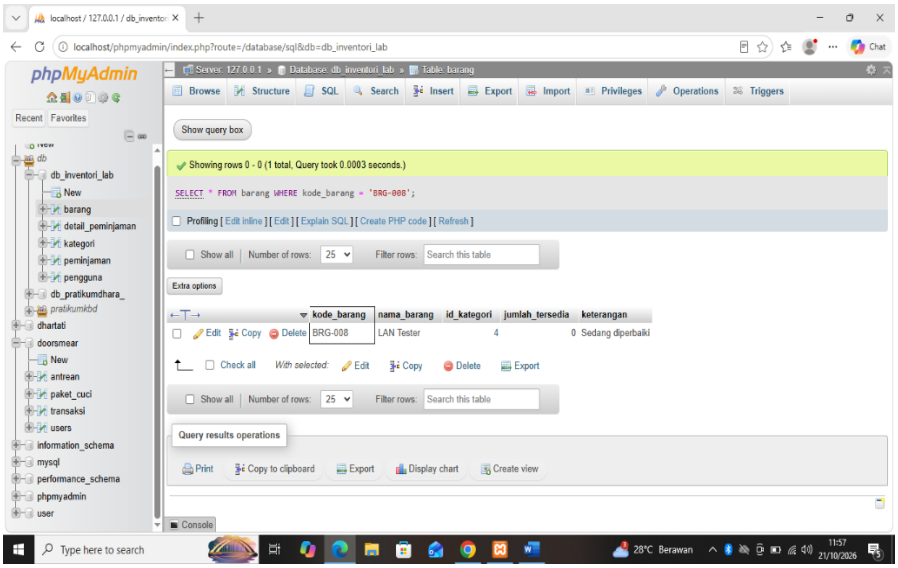
9. **melihat data sebelum update**

Sebelum mengubah data, praktik yang aman adalah melihat terlebih dahulu baris yang akan terkena dampak, memakai *SELECT* dengan *WHERE* yang sama persis dengan WHERE pada perintah UPDATE nanti. Kolom kode_barang adalah primary key, sehingga kondisi *kode_barang = 'BRG-008'* hanya menunjuk satu baris.

perintah sql

```
SELECT * FROM barang WHERE kode_barang = 'BRG-008';
```

Hasil



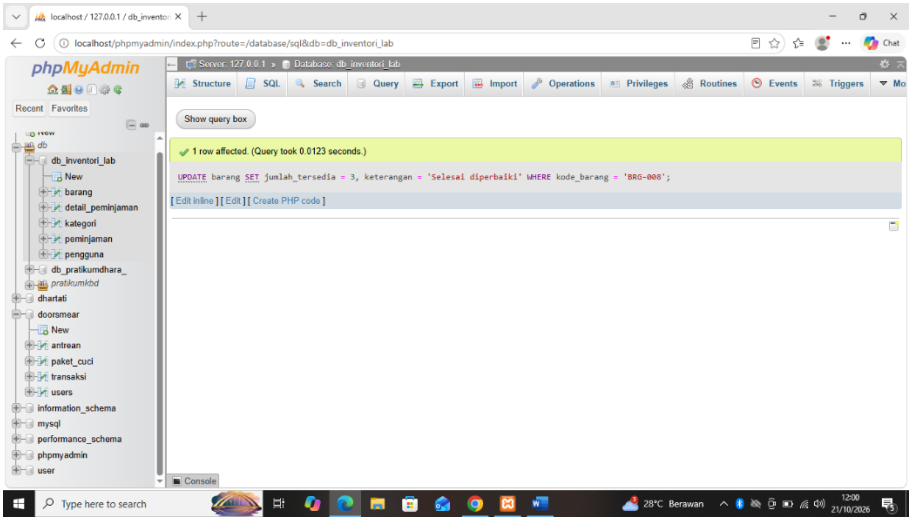
10. merubah data dengan megupdate

Perintah *UPDATE* mengubah data yang sudah ada. Klausa *SET* menentukan kolom yang diubah, yaitu jumlah_tersedia menjadi 3 dan keterangan menjadi 'Selesai diperbaiki', sedangkan *WHERE kode_barang = 'BRG-008'* membatasi perubahan hanya pada LAN Tester. Tanpa WHERE, seluruh baris pada tabel barang akan ikut berubah, sehingga WHERE pada UPDATE bersifat wajib.

perintah sql

```
UPDATE barang
SET jumlah_tersedia = 3,
    keterangan = 'Selesai diperbaiki'
WHERE kode_barang = 'BRG-008';
```

Hasil



11. Menanmbah data dan menghapus data

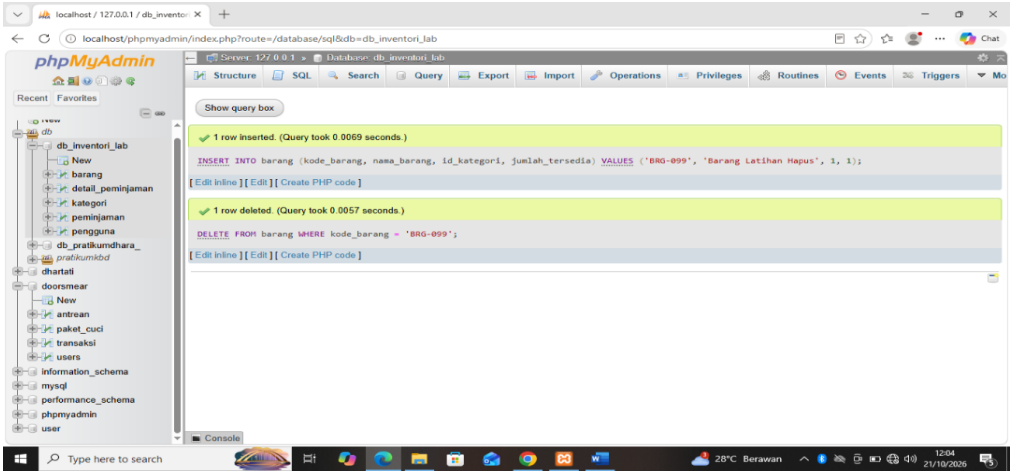
Bagian ini melatih penambahan dan penghapusan data. *INSERT INTO ... VALUES* menambahkan baris baru BRG-099 "Barang Latihan Hapus" (kategori 1, stok 1); kolom keterangan tidak diisi sehingga memakai nilai bawaan. Setelah itu *DELETE FROM barang WHERE kode_barang = 'BRG-099'* menghapus baris latihan tersebut dengan WHERE pada primary key agar hanya satu baris yang terhapus.

Perintah sql

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori, jumlah_tersedia)
VALUES ('BRG-099', 'Barang Latihan Hapus', 1, 1);

DELETE FROM barang
WHERE kode_barang = 'BRG-099';
```

Hasil



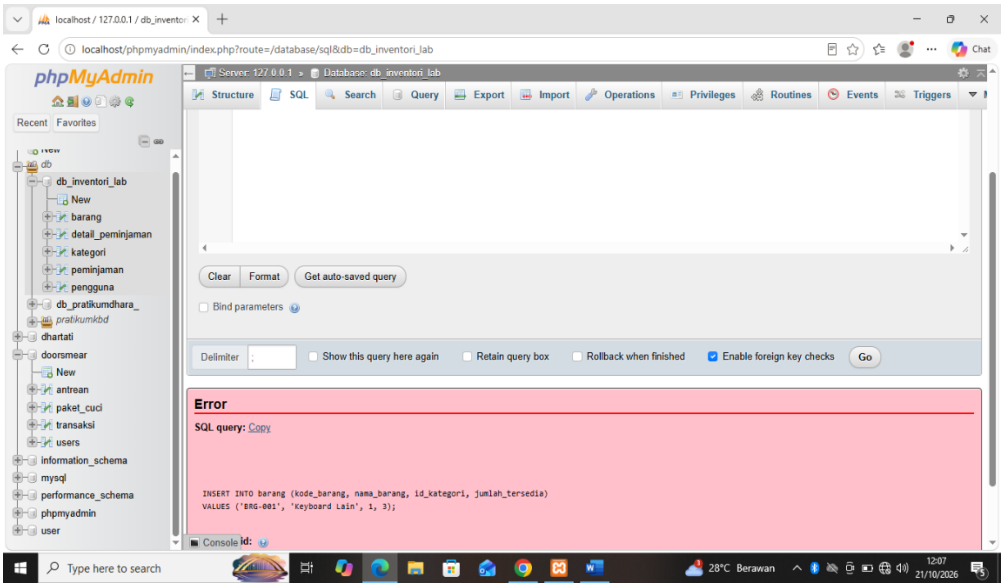
12. Menguji error 1: primary key

Mulai bagian ini, perintah yang dijalankan sengaja dibuat salah untuk membuktikan bahwa basis data menolak data yang melanggar aturan, sehingga pesan *error* justru merupakan hasil yang benar. Uji pertama mencoba memasukkan barang dengan kode BRG-001 yang sudah dipakai Keyboard USB. Kolom kode_barang adalah *PRIMARY KEY* yang harus unik, sehingga dua baris tidak boleh memiliki kode yang sama.

Perintah sql

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori, jumlah_tersedia)
VALUES ('BRG-001', 'Keyboard Lain', 1, 3);
```

Hasil



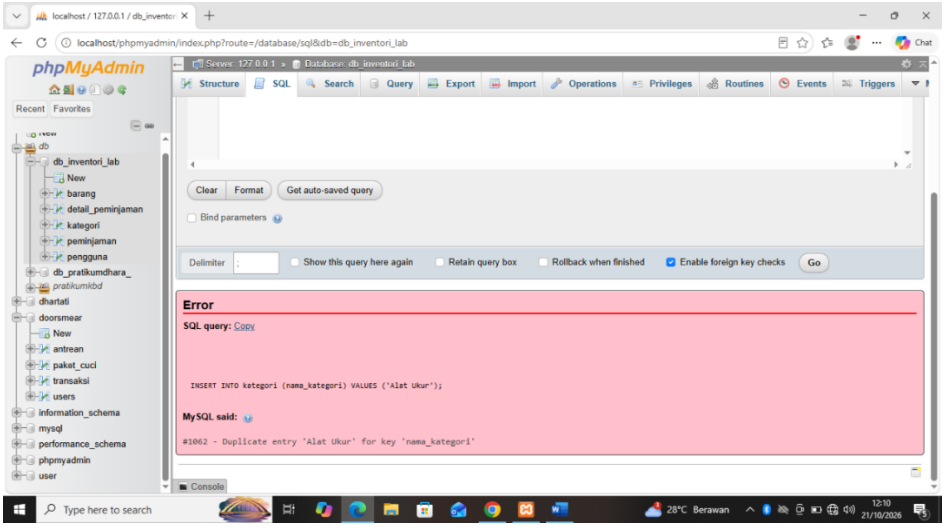
13. Uji error 2 unique

Uji kedua menambahkan kategori baru bernama 'Alat Ukur', padahal nama tersebut sudah ada. Kolom nama_kategori didefinisikan *UNIQUE*, sehingga nilainya tidak boleh kembar walaupun bukan primary key (id_kategori dibuat otomatis, jadi bisa saja berbeda).

Perintah sql

```
INSERT INTO kategori (nama_kategori) VALUES ('Alat Ukur');
```

Hasil



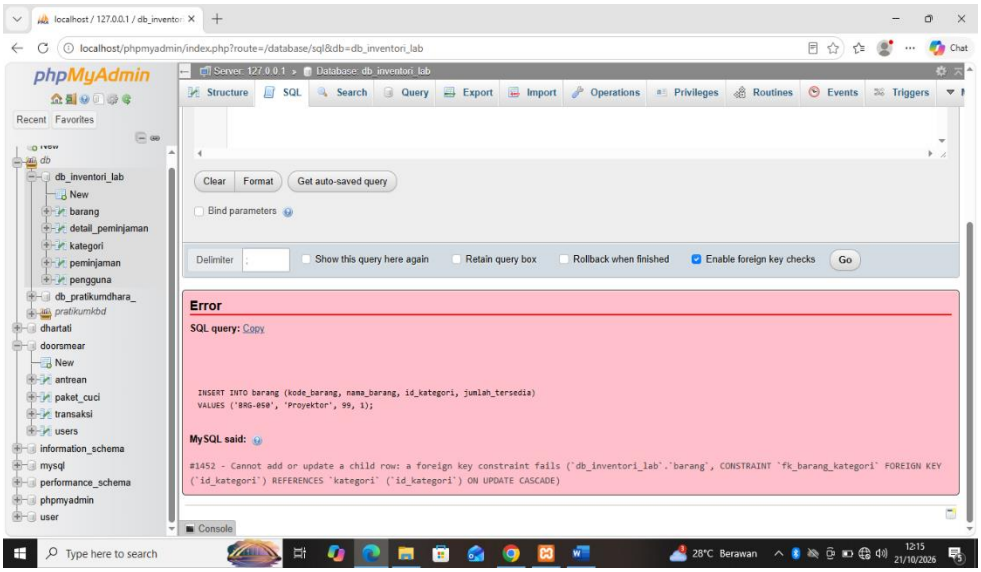
14. Uji error 3

Uji ketiga memasukkan barang BRG-050 "Proyektor" dengan id_kategori = 99, padahal tabel kategori hanya memiliki id 1 sampai 4. Kolom id_kategori pada tabel barang adalah *FOREIGN KEY* yang mengacu ke kategori(id_kategori), sehingga hanya nilai yang ada di tabel induk yang boleh dipakai.

Perintah sql

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori, jumlah_tersedia)
VALUES ('BRG-050', 'Proyektor', 99, 1);
```

Hasil



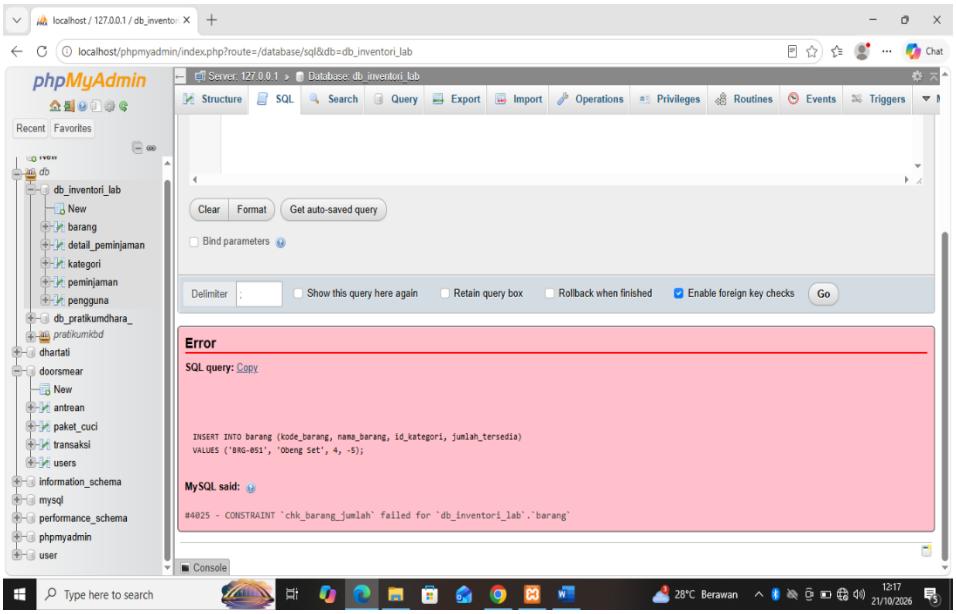
15. Uji error 4 Pelanggaran check

Uji keempat memasukkan barang BRG-051 "Obeng Set" dengan jumlah_tersedia = -5. Tabel barang memiliki aturan *CHECK* bernama chk_barang_jumlah yang melarang stok bernilai negatif, karena stok barang tidak mungkin kurang dari nol.

Perintah sql

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori, jumlah_tersedia)
VALUES ('BRG-051', 'Obeng Set', 4, -5);
```

Hasil



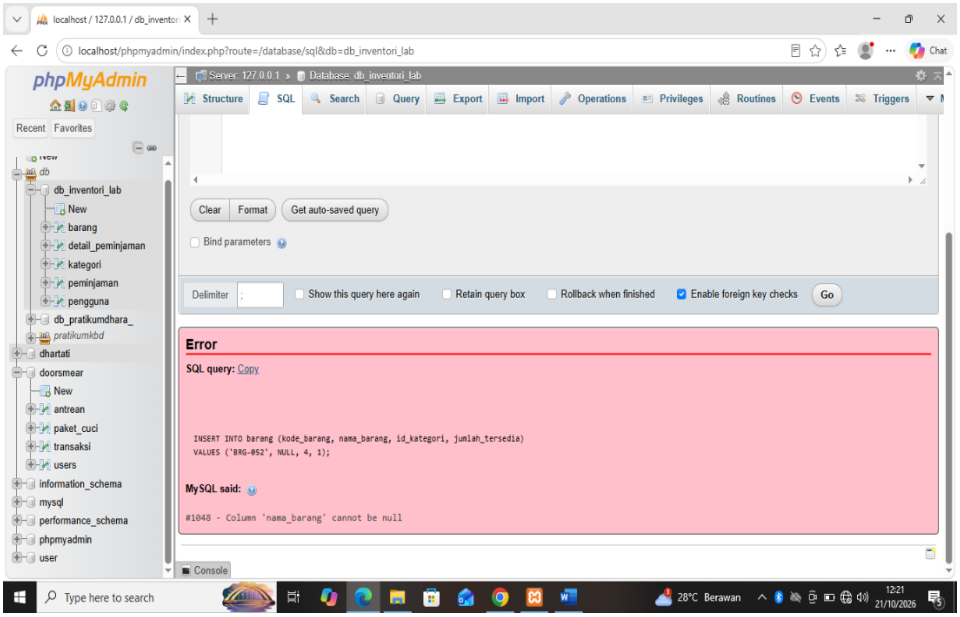
16. **Uji error 5 NULL**

Uji kelima memasukkan barang BRG-052 dengan nama_barang bernilai NULL. Kolom nama_barang dideklarasikan *NOT NULL*, artinya wajib diisi dan tidak boleh kosong.

Perintah sql

```
INSERT INTO barang (kode_barang, nama_barang, id_kategori, jumlah_tersedia)
VALUES ('BRG-052', NULL, 4, 1);
```

Hasil



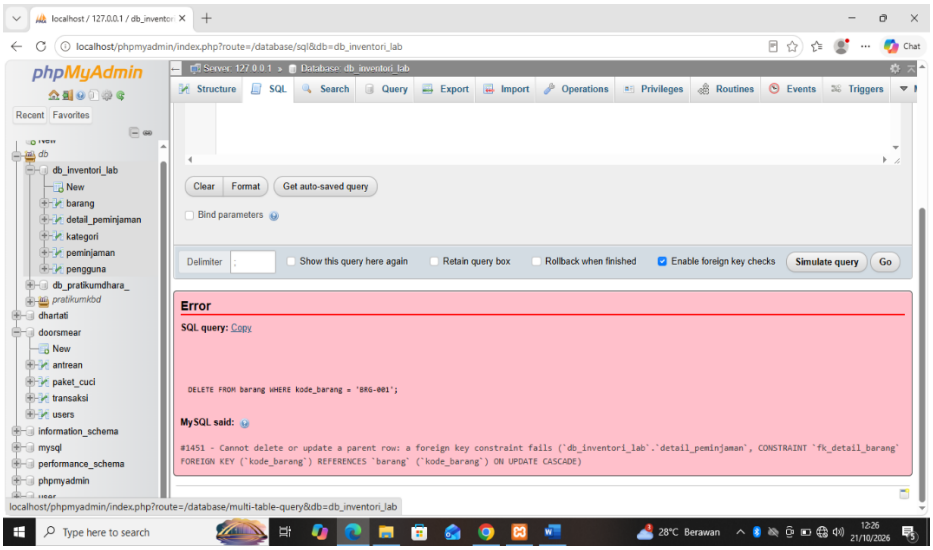
17. **Uji error 6 menghapus barang yang pernah dipinjam (ON DELETE RESTRICT)**

Uji keenam mencoba menghapus barang BRG-001 (Keyboard USB), padahal barang ini sudah tercatat pada tabel detail_peminjaman (pernah dipinjam). Foreign key fk_detail_barang pada detail_peminjaman memakai aturan *ON DELETE RESTRICT*, yaitu melarang penghapusan data induk selama masih dipakai oleh data anak.

Perintah sql

```
DELETE FROM barang WHERE kode_barang = 'BRG-001';
```

Hasil



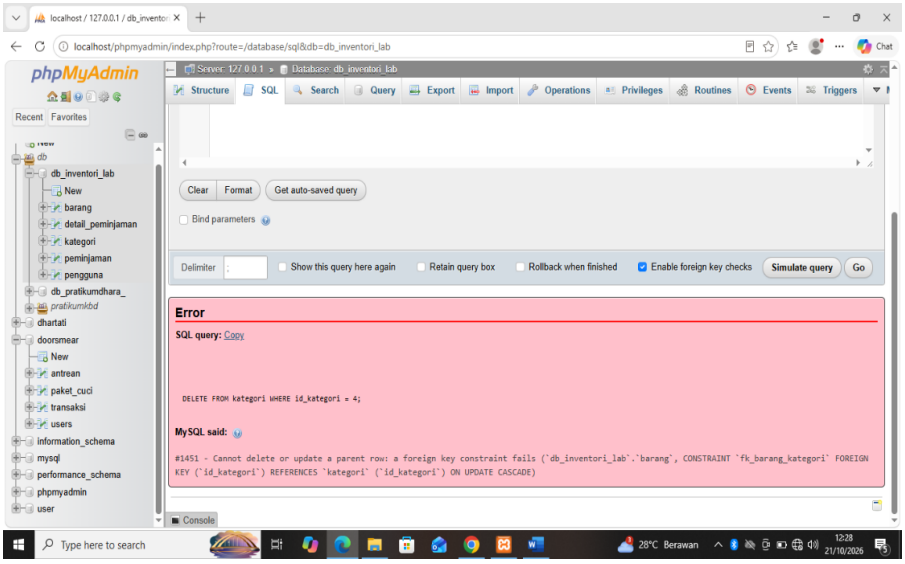
18. Uji error 7

Uji ketujuh menghapus kategori dengan id_kategori = 4 (Alat Ukur), padahal kategori tersebut masih dipakai oleh barang BRG-007 Multimeter Digital dan BRG-008 LAN Tester. Foreign key fk_barang_kategori juga menggunakan *ON DELETE RESTRICT* sehingga kategori yang masih dipakai tidak boleh dihapus.

Perintah sql

```
DELETE FROM kategori WHERE id_kategori = 4;
```

Hasil



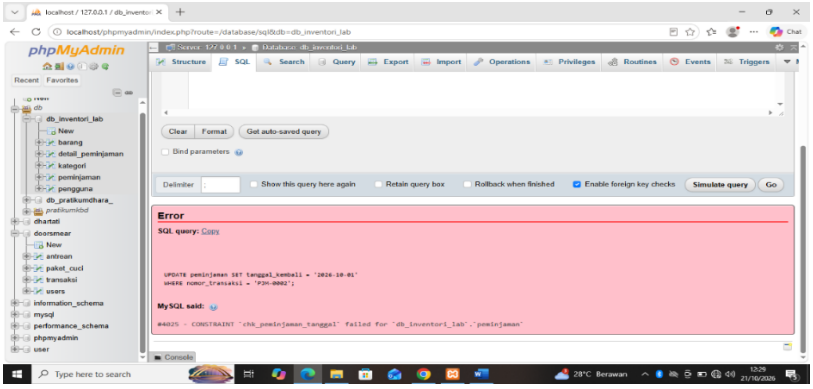
19. Uji error 8

Uji kedelapan mengubah tanggal_kembali pada transaksi PJM-0002 menjadi 2026-10-01, padahal tanggal pinjamnya adalah 2026-10-08. Tabel peminjaman memiliki aturan *CHECK* chk_peminjaman_tanggal yang mensyaratkan tanggal kembali tidak lebih awal dari tanggal pinjam, karena barang tidak mungkin dikembalikan sebelum dipinjam.

Perintah sql

```
UPDATE peminjaman SET tanggal_kembali = '2026-10-01'
WHERE nomor_transaksi = 'PJM-0002';
```

Hasil



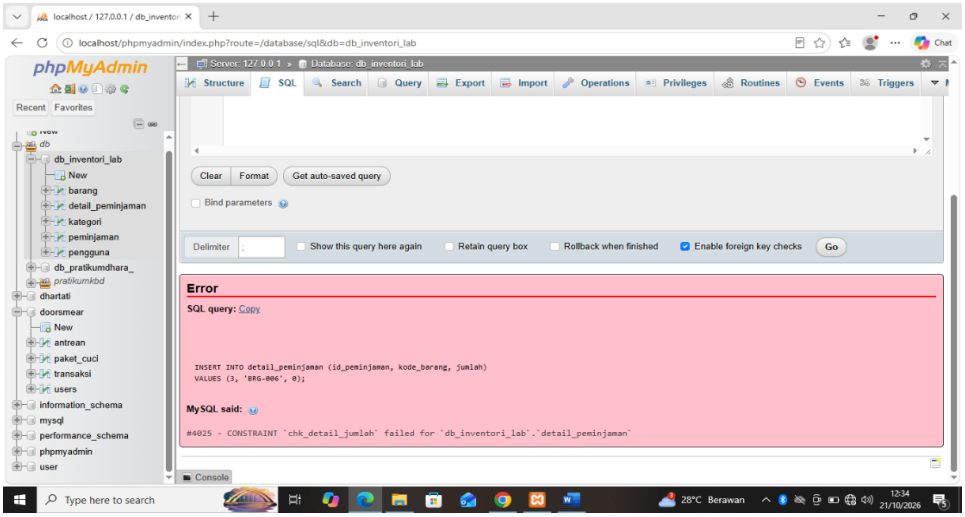
20. Uji error 9

Uji kesembilan menambahkan detail peminjaman dengan jumlah = 0 (id_peminjaman 3 dan barang BRG-006). Tabel detail_peminjaman memiliki *CHECK* chk_detail_jumlah yang mensyaratkan jumlah barang yang dipinjam harus lebih dari nol, karena meminjam nol barang tidak bermakna.

Perintah sql

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (3, 'BRG-006', 0);
```

Hasil



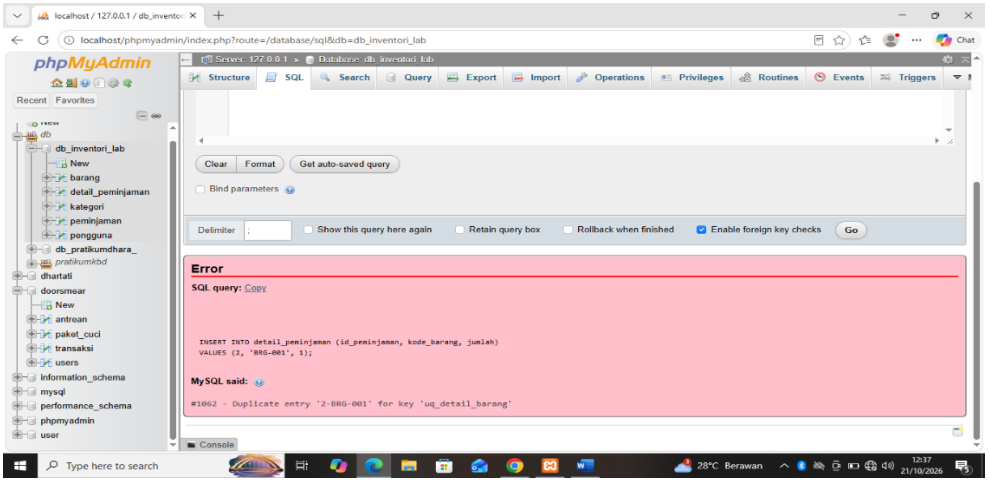
21. Uji error 10

Uji kesepuluh memasukkan kembali BRG-001 ke transaksi dengan id_peminjaman = 2, padahal pada transaksi tersebut BRG-001 sudah ada. Tabel detail_peminjaman memiliki *UNIQUE gabungan* (*composite*) bernama uq_detail_barang pada pasangan kolom (id_peminjaman, kode_barang), sehingga satu barang tidak boleh muncul dua kali dalam transaksi yang sama. Jika jumlahnya perlu ditambah, cukup lakukan UPDATE pada kolom jumlah.

Perintah sql

```
INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
VALUES (2, 'BRG-001', 1);
```

Hasil



22. Uji error 11 langkah 1

Uji kesebelas menguji aturan *ON DELETE CASCADE* pada relasi peminjaman dan detail_peminjaman. Berbeda dengan uji pertama sampai kesepuluh, uji ini seharusnya **berhasil**, dan dikerjakan dalam empat langkah

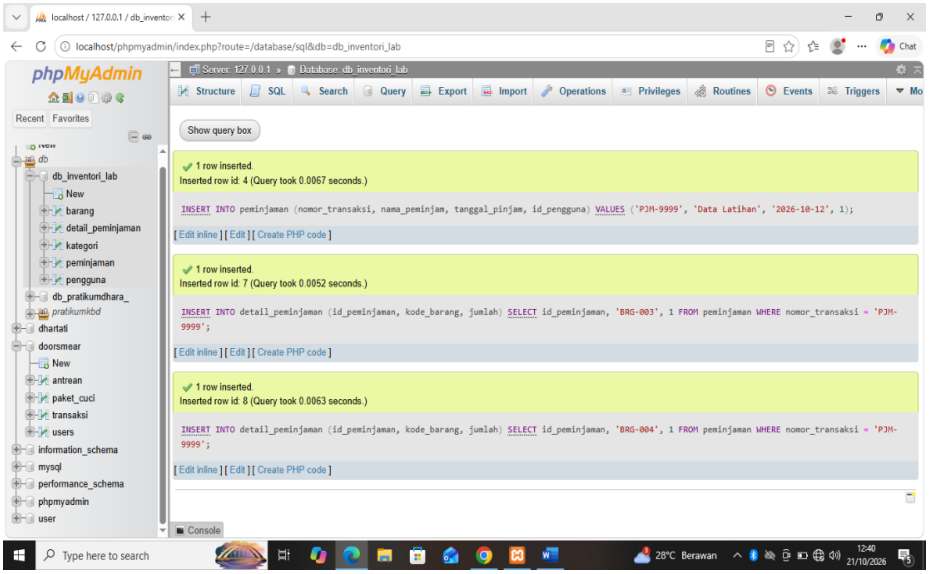
Perintah sql

```
INSERT INTO peminjaman (nomor_transaksi, nama_peminjam, tanggal_pinjam, id_pengguna)
VALUES ('PJM-9999', 'Data Latihan', '2026-10-12', 1);

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-003', 1 FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';

INSERT INTO detail_peminjaman (id_peminjaman, kode_barang, jumlah)
SELECT id_peminjaman, 'BRG-004', 1 FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';
```

Hasil



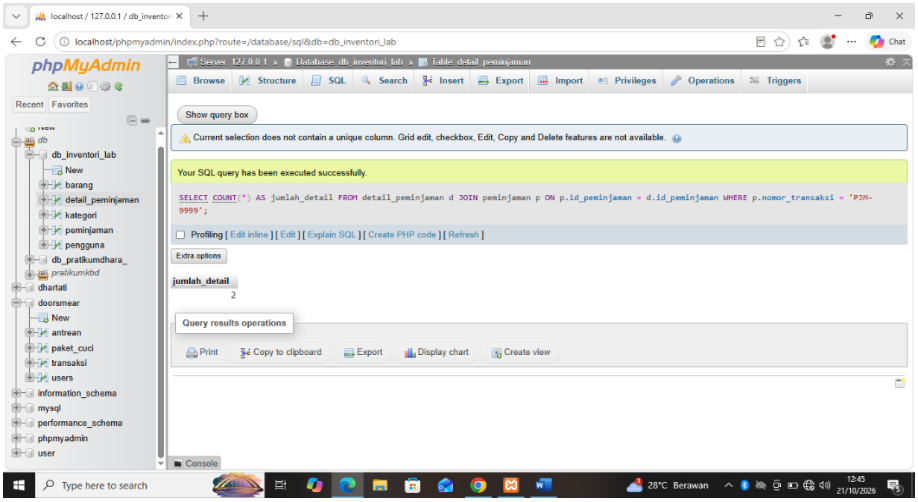
23. 11 langkah 2

Langkah 2 menghitung jumlah detail milik PJM-9999 sebelum dihapus. *COUNT() menghitung banyaknya baris, dan* JOIN* ke tabel peminjaman diperlukan karena tabel detail hanya menyimpan id_peminjaman, bukan nomor transaksi.

Perintah sql

```
SELECT COUNT(*) AS jumlah_detail
FROM detail_peminjaman d
JOIN peminjaman p ON p.id_peminjaman = d.id_peminjaman
WHERE p.nomor_transaksi = 'PJM-9999';
```

Hasil



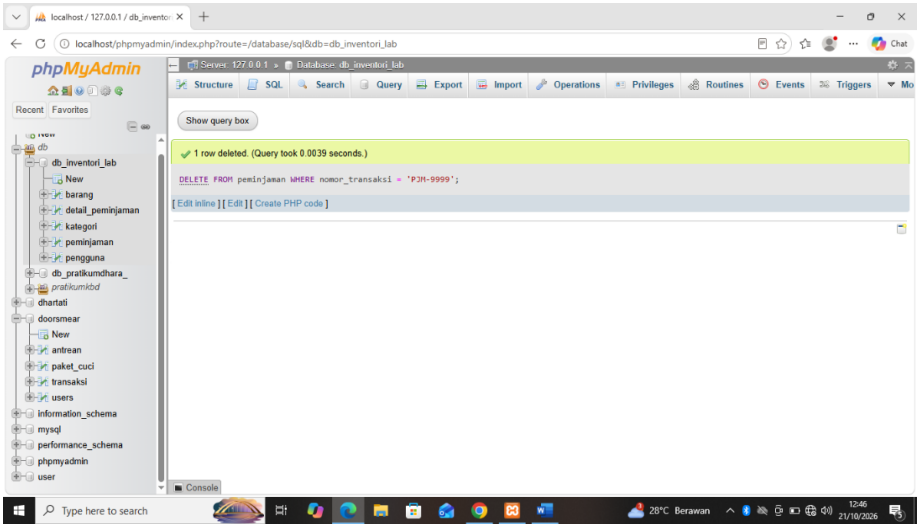
24. Uji error 11 langkah 3

Langkah 3 menghapus transaksi latihan hanya dari tabel induk peminjaman; tidak ada perintah DELETE untuk tabel detail_peminjaman. Berbeda dengan uji keenam dan ketujuh yang ditolak oleh RESTRICT, relasi ini memakai CASCADE sehingga penghapusan induk otomatis menghapus semua anaknya.

Perintah sql

```
DELETE FROM peminjaman WHERE nomor_transaksi = 'PJM-9999';
```

Hasil



25. Uji error 11 langkah 4

Langkah 4 memeriksa apakah detail barang BRG-003 masih tersisa setelah transaksi induknya dihapus, dengan menghitung baris pada detail_peminjaman yang kode_barang-nya 'BRG-003'.

Perintah sql

```
SELECT COUNT(*) AS sisa_detail_brg003
FROM detail_peminjaman
WHERE kode_barang = 'BRG-003';
```

Hasil

